

# **LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK**

*“Keamanan & Integritas Data (Enkapsulasi)”*

Disusun Oleh:

Sofian Arba'i (2511533029)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M Dawi Syauqi



**DEPARTEMEN INFORMATIKA**

**FAKULTAS TEKNOLOGI INFORMASI**

**UNIVERSITAS ANDALAS**

**2026**

## **1. Tahapan Praktikum**

Pada praktikum Pemograman Berorientasi Ojek pekan 3 dengan judul ”Keamanan & Integritas Data (Enkapsulasi)”

### **1.1 Tujuan Praktikum**

Tujuan dari dilakukan praktikum pekan 2 sebagai berikut:

1. Memahami urgensi Information Hiding dalam rekayasa perangkat lunak.
2. Mengimplementasikan Access Modifiers (private, public, protected) dengan tepat.
3. Mengontrol akses pembacaan dan penulisan atribut menggunakan Getter dan Setter.
4. Menerapkan enkapsulasi untuk memvalidasi business logic pada saat autentikasi PIN

### **1.2 Alat Praktikum**

Alat yang dibutuhkan untuk melakukan praktikum pekan 2 sebagai berikut:

1. Laptop/PC
2. JDK dan Eclipse

### **1.3 Pemahaman Teori**

Pada modul sebelumnya, semua atribut class tidak memiliki modifier, sehingga menggunakan aturan default yaitu dapat diakses langsung dari luar class. Hal ini sangat berbahaya. Jika tidak dikunci, sistem tidak akan bisa mencegah manipulasi langsung seperti `akunAktif.saldo = 9999999`.

1. Information Hiding: Robert C. Martin (2008) dalam bukunya Clean Code menekankan behavior (method), bukan mengekspos data (variabel) secara

langsung. akses data, antara lain: bahwa kita harus menyembunyikan detail implementasi internal. Objek harus mengekspos

2. Access Modifiers di Java: Java menyediakan kata kunci untuk mengatur tingkat visibilitas akses data, antara lain:

- private: Akses paling ketat. Data atau method hanya bisa diakses oleh class itu sendiri. Ini adalah aturan emas untuk melindungi state atau atribut kelas.
- protected: Akses menengah. Data atau method dapat diakses oleh class dalam package yang sama, serta oleh subclass meskipun berada di package yang berbeda. Ini sangat berguna ketika kita menerapkan hierarki Inheritance (Pewarisan).
- public: Akses paling longgar. Bisa diakses dari mana saja tanpa batasan. Umumnya digunakan untuk method layanan yang memang perlu dipanggil dari luar.

3. Getter dan Setter: Oracle (2023) menyebutkan bahwa akses ke variabel private harus difasilitasi melalui method public khusus. Ini memungkinkan kita menyisipkan logika validasi seperti cek panjang PIN sebelum variabel diproses.

## **1.4 Langkah-Langkah**

Pada langkah pertama, mengunci semua atribut yang rentan dimanipulasi diluar class dan menambahkan lapisan keamanan yaitu PIN ke dalam sistem mini bank sebelumnya.

### **1.4.1 Langkah 1 : Mengunci Entitas Transaksi**

Buka file Transaksi.java. ubah variabel menjadi private dan berikan getter untuk data yang perlu di baca. Objek transaksi bersifat immutable atau tidak boleh diubah setelah dibuat, sehingga tidak perlu membuat Setter. Dengan private, atribut ini hanya bisa diakses dari dalam class Transaksi itu sendiri. Kode di luar class (misalnya di Main.java atau

Rekening.java) tidak bisa lagi menulis `t.nominal = ....` kalau dicoba, akan error compile. Karena atribut sudah private, satu-satunya cara kode luar bisa *melihat* isi datanya adalah lewat method public khusus, yaitu getter. Getter ini bersifat read-only, getter hanya mengembalikan (return) nilai, tidak mengizinkan perubahan. Berikut kode program perubahan langkah 1

```
public class Transaksi {
    private String idTransaksi;
    private String jenis;
    private double nominal;

    //constructor
    public Transaksi (String id, String jenis, double
nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    // 2. Hanya Menyediakan Getter (Read Only)
    public String getIdTransaksi () {
        return idTransaksi;
    }
    public String getJenis() {
        return jenis;
    }
    public double getNominal() {
        return nominal;
    }

    public void cetakDetail () {
        System.out.println("ID: " + idTransaksi + " |
Jenis: " + jenis + " | Nominal : " + nominal);
    }
}
```

Kode program di atas dibagi menjadi beberapa bagian sebagai berikut:

#### 1. Deklarasi Class dan Atribut

Class Transaksi punya 3 atribut, semuanya bermodifiler private. Berarti ketiga atribut ini hanya bisa diakses langsung dari dalam class Transaksi sendiri. Tidak ada class lain (seperti Rekening atau Main)

yang bisa langsung menulis atau membaca atribut ini tanpa lewat method yang disediakan pada class ini. Ini adalah bentuk enkapsulasi (information hiding).

## 2. Constructor

Constructor public Transaksi, dipanggil satu kali saat objek Transaksi baru dibuat, misalnya `new Transaksi ("TRX-001", "Kredit", 50000)`. Nilai dari parameter (id, jenis, nominal) disalin ke atribut objek (`this.idTransaksi`, `this.jenis`, `this.nominal`). Kata kunci `this` dipakai untuk membedakan atribut milik objek dengan parameter yang kebetulan namanya mirip/sama.

## 3. Getter (Read-Only)

Public String `getIdTransaksi`, `getJenis`, `getNominal`, Karena atribut private, getter adalah satu-satunya cara kode dari luar bisa membaca nilainya, misalnya `trx.getNominal()`. Getter ini hanya return, tidak ada logika untuk mengubah data. Tidak ada setter karena sebuah catatan transaksi seharusnya menjadi fakta permanen/riwayat yang tidak boleh dimanipulasi lagi setelah tercatat.

## 4. Method cetakDetail()

Method ini mencetak semua informasi transaksi dalam satu baris. Karena method ini ada di dalam class Transaksi itu sendiri, ia boleh langsung memakai atribut private (`idTransaksi`, `jenis`, `nominal`) tanpa perlu memanggil getter-nya sendiri.

### 1.4.2 Langkah 2 : Mengunci Entitas Rekening dan Menambahkan PIN

Langkah selanjutnya adalah mengamankan saldo dan menambahkan atribut PIN. Berikut kode program pada class `Rekening.java`:

```

import java.util.ArrayList;
public class Rekening {
    //1. mengunci atribut dengan private
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; //Data sensitif

    // percobaan PIN salah & status blokir
    private int percobaanGagal;
    private boolean isBlocked;

    // Implementasi Asosiasi (1-to-many)
    private ArrayList<Transaksi> riwayatTransaksi;

    //2. Modifikasi Constructor untuk menerima PIN awal
    public Rekening(String nomor, String nama, double
saldoAwal, String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di dalam Constructor
        if (pinAwal.length()==6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus
6 digit! Menggunakan PIN default 123456");
            this.pin = "123456";
        }

        // Wajib menginisialisasi ArrayList di dalam
constructor agar tidak NullPointerException
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " +
namaPemilik + " berhasil dibuat dengan saldo Rp" +
saldo);
    }

    // 3. Getter untuk atribut yang diizinkan
    public String getNomorRekening() {
        return nomorRekening;
    }

    public String getNamaPemilik() {
        return namaPemilik;
    }

    // 4. Method Otentikasi Internal (Validasi
Enkapsulasi)
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }
}

```

Kode program di atas terbagi menjadi beberapa bagian seperti deklarasi atribut, constructor dll. Berikut pembagian penjelasan kode program penambahan.

### 1. Deklarasi Atribut

Semua atribut dikunci private. pin diberi komentar "Data sensitif" karena memang tidak boleh dibaca dari luar sama sekali. riwayatTransaksi adalah bentuk relasi *1-to-many* (satu rekening punya banyak transaksi).

### 2. Constructor

Mulai dari public rekening-riwayatTransaksi, Constructor menyimpan data dasar rekening, lalu memvalidasi PIN: jika panjangnya persis 6 karakter maka dipakai, jika tidak akan diganti otomatis ke "123456" disertai pesan peringatan. riwayatTransaksi diinisialisasi sebagai ArrayList kosong agar siap diisi transaksi nanti. Hanya dua atribut public String getNomorRekening dan getNamaPemilik yang dibaca dari luar. Data seperti saldo dan pin sengaja tidak punya getter langsung karena saldo harus diakses lewat method seperti cekInformasi(), dan pin tidak boleh terlihat sama sekali (kalau ada getPin(), siapa pun bisa mengintip PIN orang lain).

### 3. Method otentikasi (boolean)

Method ini membandingkan PIN yang di-input dengan PIN asli yang tersimpan di objek. Karena pin bersifat private, satu-satunya cara memvalidasi PIN dari luar class adalah lewat method ini. Kode luar tidak pernah tahu isi PIN yang sebenarnya, hanya tahu hasilnya true atau false. Ini inti dari enkapsulasi: logika validasi disembunyikan di dalam class.

## 5. Laporan Tugas/Latihan

Pada pertemuan praktikum pekan 3, sistem menuntut PIN saat pembuatan akun dan memiliki fungsi otentifikasi ().

Tugas : Mengamankan CLI dengan Otentifikasi

1. Menu 1 (Buka Rekening): Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).

Pada tugas 1 diminta untuk menambahkan aturan input password saat membuat akun. Kode program untuk tugas 1 pada bagian sebagai berikut:

```
case 1:
System.out.print("Masukkan No Rekening: ");
String no = input.nextLine();
System.out.print("Masukkan Nama Pemilik: ");
String nama= input.nextLine();
System.out.print("Masukkan Saldo Awal: ");
if (!input.hasNextDouble()) {
    System.out.println("Error: Saldo awal harus berupa
angka! Pembuatan rekening dibatalkan.");
    input.next();
    break;
}
double saldo = input.nextDouble();
input.nextLine();

System.out.print("Masukkan PIN (6 digit): ");
String pinAwal = input.nextLine();

akunAktif = new Rekening(no, nama, saldo, pinAwal);
daftarRekening.add(akunAktif);
break;

    public String getNamaPemilik() {
        return namaPemilik;
    }

    // 4. Method Otentikasi Internal (Validasi
    Enkapsulasi)
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }
```

```

===== SISTEM PERBANKAN MINI =====
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu:
1
Masukkan No Rekening: 12345
Masukkan Nama Pemilik: Sofian
Masukkan Saldo Awal: 100000
Masukkan PIN (6 digit): 221006

```

Penjelasan kode program untuk tugas 1, pertama yaitu meminta user memasukkan nomor rekening, nama pemilik, dan saldo awal. Sebelum membaca saldo, dicek dulu apakah input berikutnya benar-benar berupa angka desimal (`hasNextDouble()`). Kalau bukan angka, program mencetak pesan error.

Selanjutnya yaitu menginputkan PIN untuk setiap rekening yang dibuat. Instansiasi Objek dan penyimpanan dengan mengumpulkan (no, nama, saldo, pinAwal) dikirim ke constructor Rekening untuk membuat objek baru. Objek baru ini langsung dijadikan `akunAktif` (akun yang sedang aktif digunakan), sekaligus dimasukkan ke `daftarRekening` (daftar semua rekening yang pernah dibuat).

2. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.

- Panggil fungsi `akunAktif.otentikasi(pinYangDiinput)`.
- Jika mengembalikan nilai `true`, jalankan transaksi.
- Jika `false`, batalkan dan cetak: "Akses Ditolak: PIN yang Anda masukkan salah!".

Pada tugas 2 diminta menambahkan sistem keamanan pada transaksi dengan cara sebelum sistem mengeksekusi menu 3 (Tarik Tunai) dan 6 (Cetak Mutasi), user diminta untuk memasukkan PIN seperti saat pembuatan rekening sebelumnya.

a. Menu 3 (Tarik Tunai)

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon Maaf, Anda
        belum memiliki nomor rekening!");
    } else if (akunAktif.isBlocked()) {
        System.out.println("Akun Anda telah diblokir
        karena 3x salah PIN. Transaksi tidak dapat
        dilakukan.");
    } else {
        System.out.print("Masukkan PIN: ");
        String pinTarik = input.nextLine();

        if (akunAktif.otentikasi(pinTarik)) {
            System.out.print("Masukkan Nominal Tarik:
            ");
            double Tarik = input.nextDouble();
            input.nextLine();
            akunAktif.tarikTunai(Tarik);
        } else {
            System.out.println("Akses Ditolak: PIN
            yang Anda masukkan salah!");
            if (akunAktif.isBlocked()) {
                System.out.println("Akun Anda sekarang
                TERBLOKIR karena 3x salah PIN.");
            }
        }
    }
    break;
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu:
3
Masukkan PIN: 221006
Masukkan Nominal Tarik: 15000
Tarik tunai Rp15000.0 berhasil. Saldo saat ini: Rp85000.0
```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu:
3
Masukkan PIN: 123456
Akses Ditolak: PIN yang Anda masukkan salah!

```

Pertama cek apakah ada akun aktif, lalu cek status blokir pada akun, jika akun sudah diblokir maka tidak akan bisa untuk tarik tunai langsung. Lalu sistem meminta user untuk menginputkan PIN seperti intruksi tugas lalu memanggil `akunAktif.otentikasi(pinTarik)`, jika PIN benar masukkan nominal yang ingin ditarik. Jika PIN salah maka akan menampilkan pesan `System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");`.

#### b. Menu 6 (Cek Mutasi)

```

case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening ");
    } else {
        System.out.print("Masukkan PIN: ");
        String pinMutasi = input.nextLine();

        if (akunAktif.otentikasi(pinMutasi)) {
            System.out.println("=== MUTASI REKENING ===");
            akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
            if (akunAktif.isBlocked()) {
                System.out.println("Akun Anda sekarang TERBLOKIR karena 3x salah PIN.");
            }
        }
    }
    break;

```

Penjelasan kode program, pertama cek apakah ada akunAktif, jika ada masukkan PIN yang sama seperti saat pembuatan akun pada awal program. Lalu memanggil otentikasi akunAktif.otentikasi(pinMutasi). Jika PIN mutase yang dimasukkan benar, cetak mutasi rekening dengan memanggil akunAktif.cetakMutasi (). Jika PIN yang yang dimasukkan salah cetak pesan error ” Akses Ditolak: PIN yang Anda masukkan salah!”.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu:
6
Masukkan PIN: 123456
Akses Ditolak: PIN yang Anda masukkan salah!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu:
6
Masukkan PIN: 221006
=== MUTASI REKENING ===
ID: TRX-T-1790364406991 | Jenis: Debit | Nominal :15000.0
```

## 6. Link Repository

Berikut link github untuk repository praktikum pekan 3:

[sofianarbai/PrakPBO\\_C\\_2511533029](https://github.com/sofianarbai/PrakPBO_C_2511533029)